

Extending the Linear Model 6: Generalised Linear Models Lab

Dr. Luisa Barbanti and Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1	Load packages	2
2	Count data	2
2.1	Introductory example	2
2.2	Poisson model	5
2.3	The Binomial Model in R	11
3	Further on Binary and Binomial Data	14
3.1	Binary data: Logistic Regression	14
3.2	Multinomial Data	14
3.3	Estimating the Performance of a Binary Model	15
4	Appendix	20
4.1	Interpreting the Coefficients of a Poisson Model	20
4.1.1	Interpreting the Coefficients of a Poisson Model: Factors	20
4.1.2	Interpreting the Coefficients of a Poisson Model: Continuous Predictors	21
4.2	Interpreting the Coefficients of a Binomial Model	22
4.2.1	Interpreting the Coefficients of a Binomial Model: Factors	25

1 Load packages

```
## Extending the Linear Model 6: Generalised Linear Models Lab
## Load packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.formula.api import ols, glm
from statsmodels.discrete.discrete_model import Poisson, Logit
from statsmodels.genmod.generalized_linear_model import GLM
from statsmodels.genmod.families import Binomial, Poisson as PoissonFamily#, QuasiPoisson
from statsmodels.gam.api import GLMGam, BSplines
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import confusion_matrix
from scipy.special import expit
from sklearn.datasets import load_iris
from pygam import LogisticGAM, s
```

2 Count data

2.1 Introductory example

```
## Set seed for reproducibility
np.random.seed(1)

## Simulate data for smokers
no_days = 14
v_non_smoker = np.concatenate([np.random.poisson(lam = 0, size = no_days - 1), [1]])
v_smoker1_2 = np.random.poisson(lam = 5, size = no_days)
v_smoker_box = np.random.poisson(lam = 20, size = no_days)

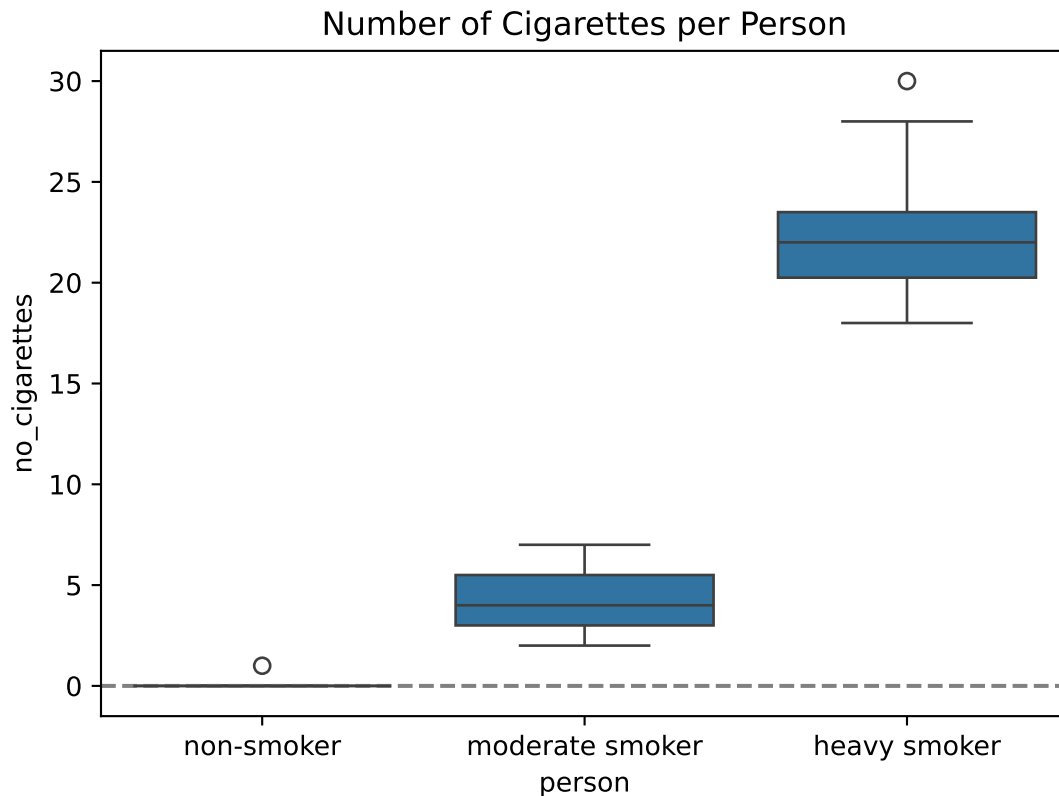
d_smokers = pd.DataFrame({
    'no_cigarettes': np.concatenate([v_non_smoker, v_smoker1_2, v_smoker_box]),
    'person': np.repeat(['non-smoker', 'moderate smoker', 'heavy smoker'], no_days)
})
d_smokers.head()
```

	no_cigarettes	person
0	0	non-smoker
1	0	non-smoker
2	0	non-smoker
3	0	non-smoker
4	0	non-smoker

```

## Clean figure object
plt.clf()
## Plotting with seaborn
sns.boxplot(data = d_smokers, x = 'person', y = 'no_cigarettes')
plt.axhline(0, color = 'gray', linestyle = '--')
plt.title('Number of Cigarettes per Person')

```



```

## Modelling count data with a linear model
lm_smokers = ols('no_cigarettes ~ person', data = d_smokers).fit()
print("Linear Model Coefficients:")

```

Linear Model Coefficients:

```
print(np.round(lm_smokers.params, 1))
```

```

Intercept                22.4
person[T.moderate smoker] -18.1
person[T.non-smoker]      -22.3
dtype: float64

```

```

## Simulate observations from the linear model
np.random.seed(3)
sim_data_smokers = lm_smokers.predict(d_smokers) + np.random.normal(0,

```

```
lm_smokers.resid.std(),  
size = len(d_smokers))
```

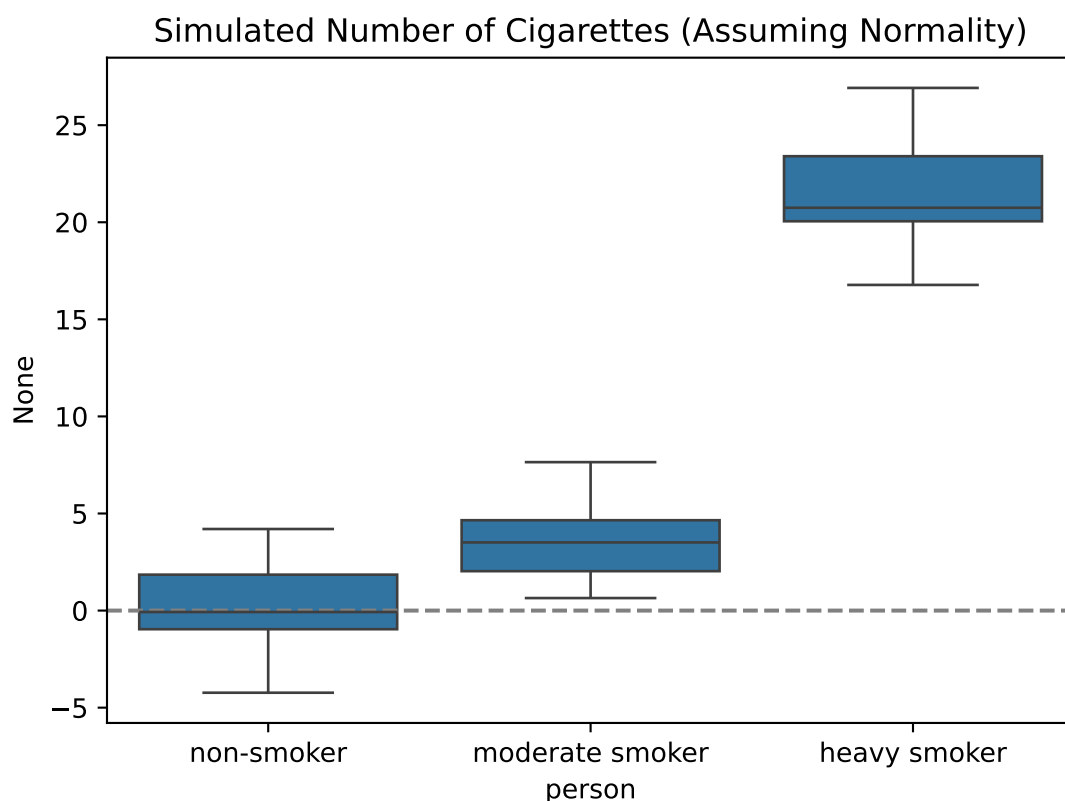
```
sim_data_smokers.head()
```

```
0    4.200515  
1    1.079121  
2    0.294195  
3   -4.230483  
4   -0.568928  
dtype: float64
```

```
sim_data_smokers.tail()
```

```
37    19.993504  
38    24.951869  
39    22.052616  
40    18.609755  
41    23.850007  
dtype: float64
```

```
## Plot simulated data  
plt.clf()  
sns.boxplot(x = d_smokers['person'], y = sim_data_smokers)  
plt.axhline(0, color = 'gray', linestyle = '--')  
plt.title('Simulated Number of Cigarettes (Assuming Normality)')
```



2.2 Poisson model

```
## Poisson GLM
glm_smokers = smf.glm('no_cigarettes ~ person',
                     data = d_smokers,
                     family = PoissonFamily()).fit()
print("Poisson GLM Summary:")
```

Poisson GLM Summary:

```
print(glm_smokers.summary())
```

```
=====
Generalized Linear Model Regression Results
=====
Dep. Variable:          no_cigarettes    No. Observations:          42
Model:                  GLM              Df Residuals:              39
Model Family:           Poisson          Df Model:                  2
Link Function:           Log              Scale:                    1.0000
Method:                  IRLS             Log-Likelihood:           -69.150
Date:                    Tue, 07 Oct 2025 Deviance:                  21.674
Time:                    13:49:09          Pearson chi2:              29.7
No. Iterations:          8                 Pseudo R-squ. (CS):       1.000
Covariance Type:         nonrobust
```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	3.1071	0.057	54.971	0.000	2.996	3.218
person[T.moderate smoker]	-1.6687	0.142	-11.757	0.000	-1.947	-1.390
person[T.non-smoker]	-5.7462	1.002	-5.737	0.000	-7.709	-3.783

```
## Simulate Poisson data from GLM
```

```
np.random.seed(2)
sim_data_smokers_poisson = np.random.poisson(lam = glm_smokers.predict(d_smokers))

sim_data_smokers_poisson_df = pd.DataFrame({'simulated_y': sim_data_smokers_poisson})
sim_data_smokers_poisson_df.head()
```

```

    simulated_y
0             0
1             0
2             0
3             0
4             0

```

```
sim_data_smokers_poisson_df.tail()
```

```

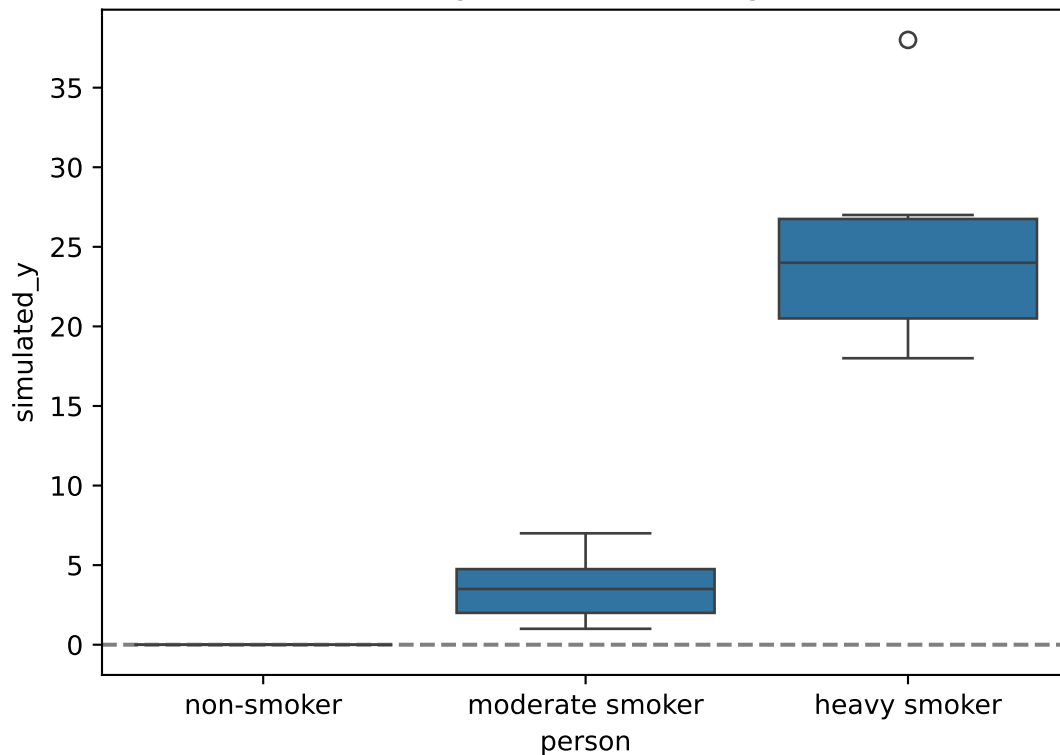
    simulated_y
37            24
38            27
39            27
40            24
41            19

```

```
## Plot simulated Poisson data
```

```
plt.clf()
sns.boxplot(x = d_smokers['person'], y = sim_data_smokers_poisson_df['simulated_y'])
plt.axhline(0, color = 'gray', linestyle = '--')
plt.title('Simulated Number of Cigarettes (Assuming Poisson Distribution)')
```

Simulated Number of Cigarettes (Assuming Poisson Distribution)



Reproduce bliss data set

```
bliss = pd.DataFrame({
    "dead": [2, 8, 15, 23, 27],
    "alive": [28, 22, 15, 7, 3],
    "conc": [0, 1, 2, 3, 4]
})

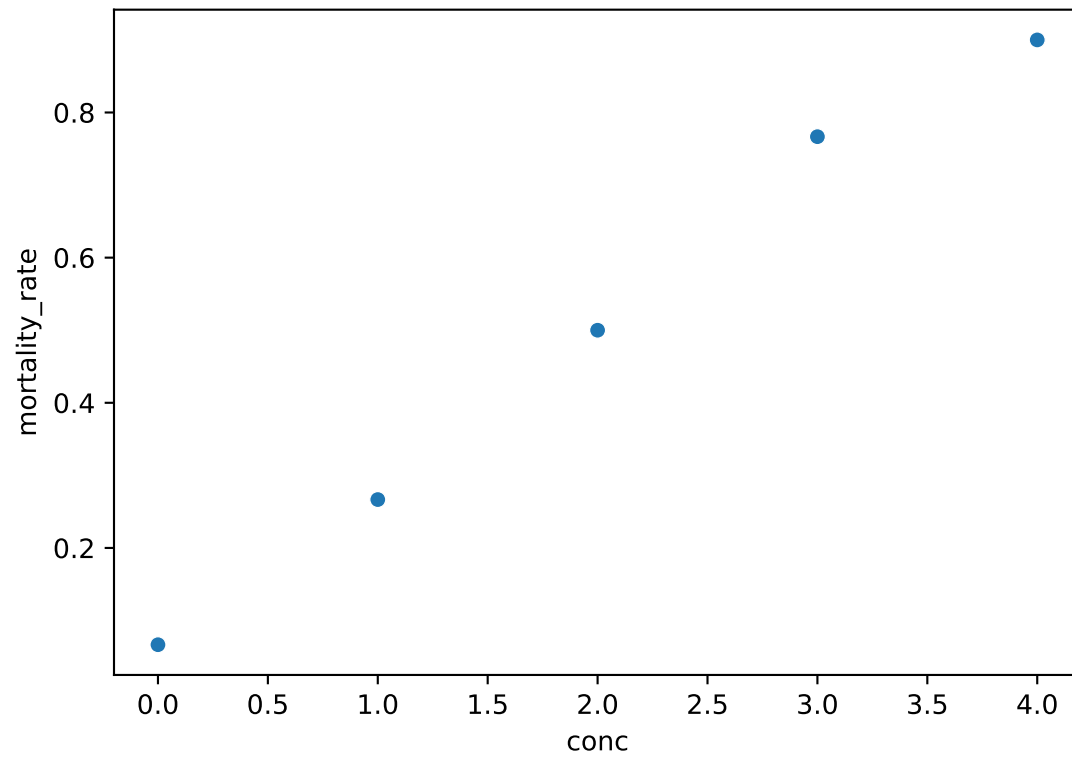
bliss["total_insects"] = bliss["dead"] + bliss["alive"]
bliss["mortality_rate"] = bliss["dead"] / bliss["total_insects"]

print(bliss)
```

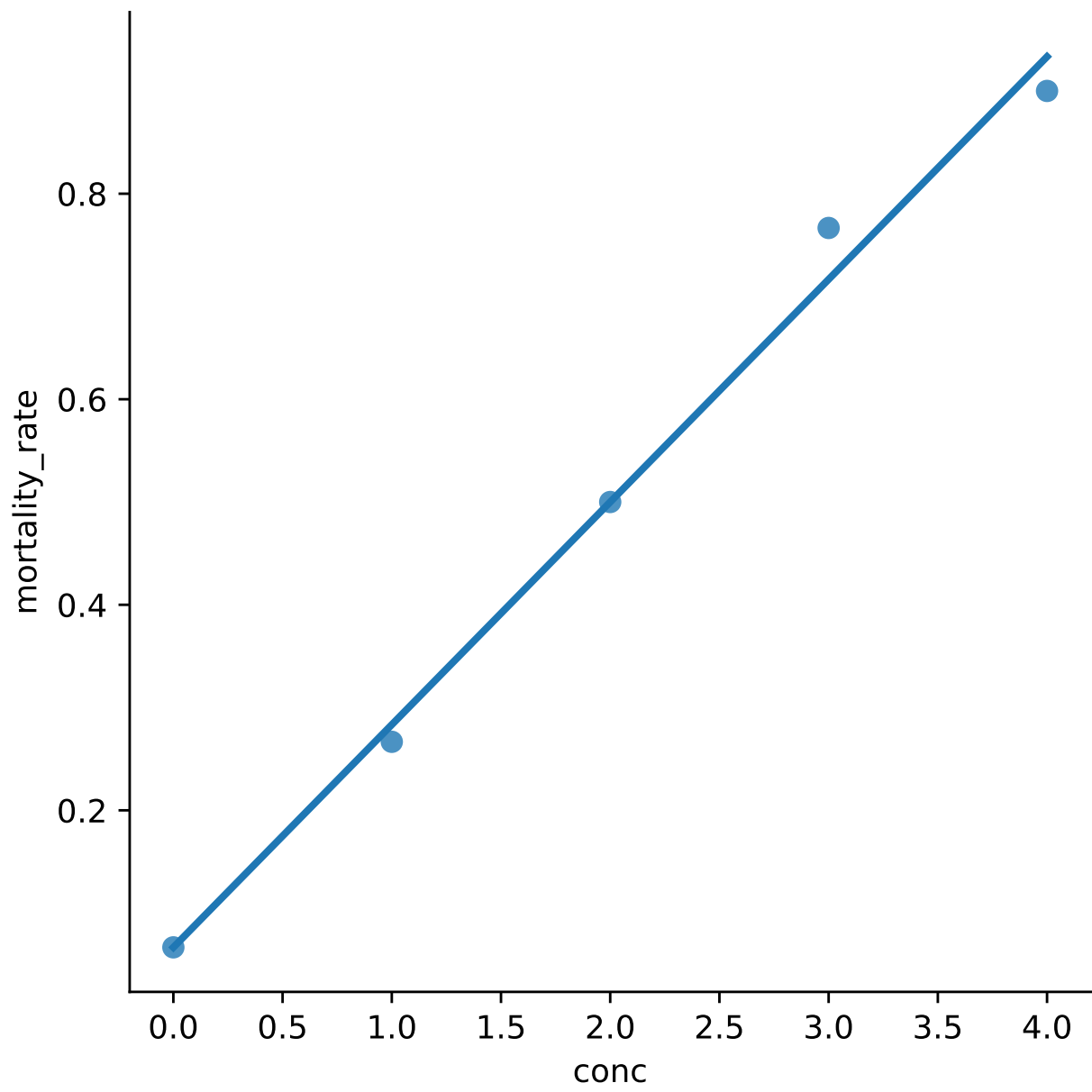
	dead	alive	conc	total_insects	mortality_rate
0	2	28	0	30	0.066667
1	8	22	1	30	0.266667
2	15	15	2	30	0.500000
3	23	7	3	30	0.766667
4	27	3	4	30	0.900000

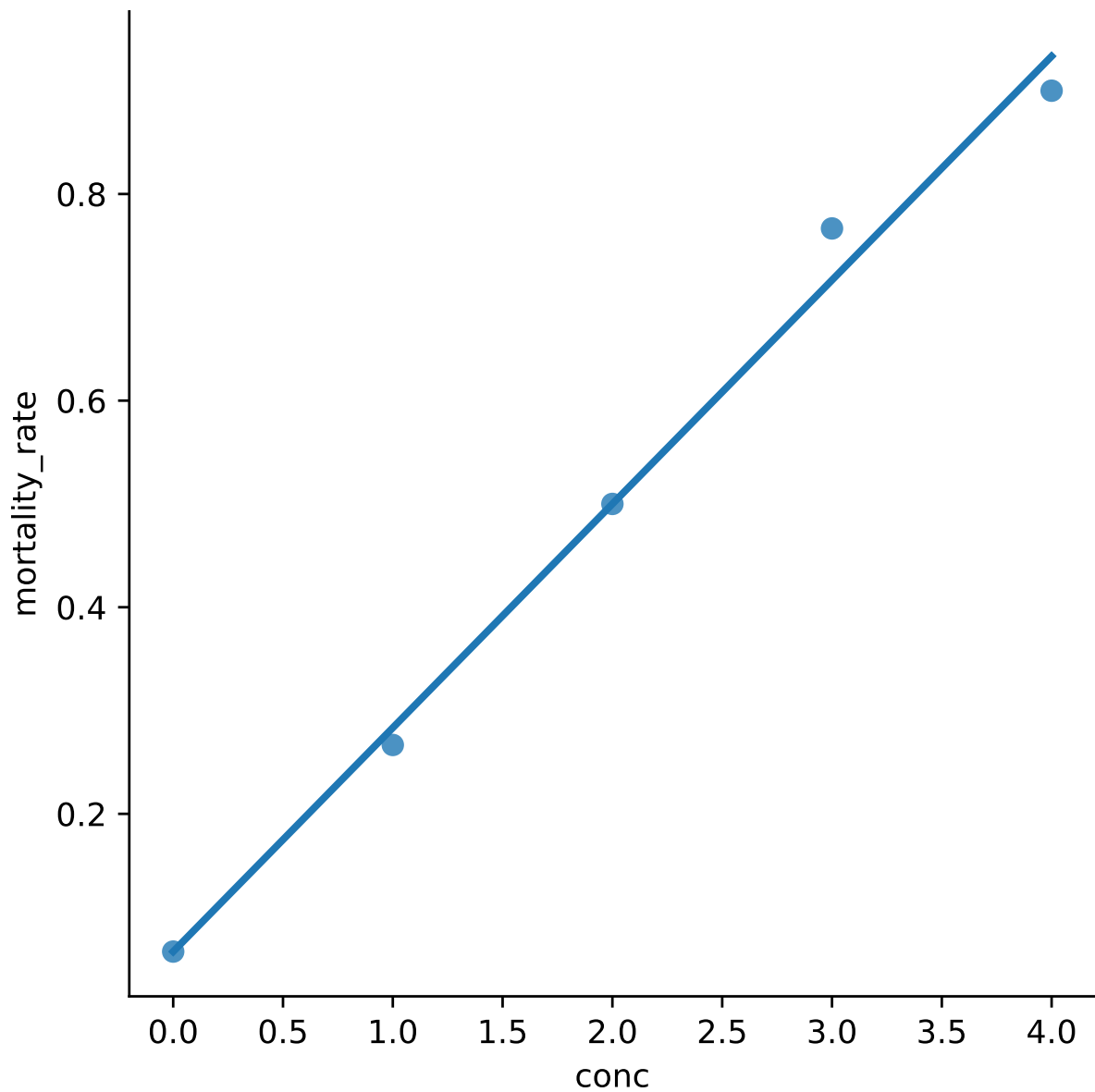
Plot simulated Poisson data

```
plt.clf()
sns.scatterplot(x = bliss['conc'], y = bliss['mortality_rate'])
```



```
plt.clf()
sns.scatterplot(x = 'conc', y = 'mortality_rate', data = bliss)
sns.lmplot(x = 'conc', y = 'mortality_rate', data = bliss, ci = None)
```



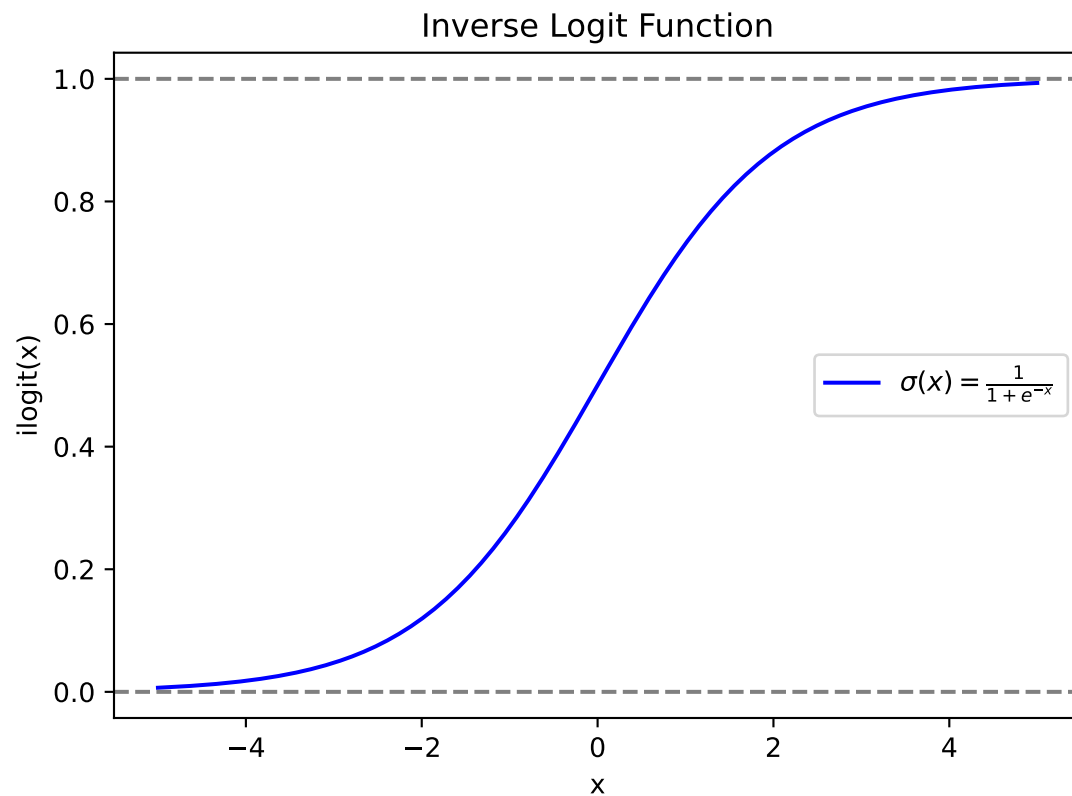
```
plt.clf()
# Generate x values from -5 to 5
x = np.linspace(-5, 5, 300)

# Compute the inverse logit function (logistic sigmoid)
y = expit(x)

# Plot the curve
plt.plot(x, y, label = r'$\sigma(x) = \frac{1}{1 + e^{-x}}$', color = 'blue')

# Add horizontal reference lines at y = 0 and y = 1
plt.axhline(y = 0, color = 'gray', linestyle = '--')
plt.axhline(y = 1, color = 'gray', linestyle = '--')
```

```
# Labels and title
plt.xlabel("x")
plt.ylabel("ilogit(x)")
plt.title("Inverse Logit Function")
plt.legend()
```



2.3 The Binomial Model in R

```
glm_insects = smf.glm("mortality_rate ~ conc",
                      data = bliss,
                      family = sm.families.Binomial()).fit()
print(glm_insects.summary())
```

```
=====
Generalized Linear Model Regression Results
=====
Dep. Variable:      mortality_rate    No. Observations:      5
Model:              GLM               Df Residuals:          3
Model Family:       Binomial          Df Model:              1
Link Function:      Logit             Scale:                1.0000
Method:             IRLS              Log-Likelihood:        -1.6358
Date:               Tue, 07 Oct 2025   Deviance:              0.012625
Time:               13:49:09           Pearson chi2:          0.0122
```

No. Iterations: 5 Pseudo R-squ. (CS): 0.3490
Covariance Type: nonrobust

	coef	std err	z	P> z	[0.025	0.975]
Intercept	-2.3238	2.289	-1.015	0.310	-6.810	2.162
conc	1.1619	0.994	1.169	0.242	-0.786	3.109

```
plt.clf()
# Generate new data for prediction
new_data = pd.DataFrame({"conc": np.linspace(0, 5, 100)})
new_data["pred_insects"] = glm_insects.predict(new_data)

# Plot
plt.figure(figsize = (8, 6))
plt.grid()

# Plot predictions
sns.scatterplot(x = new_data["conc"], y = new_data["pred_insects"],
               color = "blue", s = 50, label = "Predictions")

# Plot actual observations on top
sns.scatterplot(x = bliss["conc"], y = bliss["mortality_rate"],
               color = "red", s = 100, label = "Actual observations")

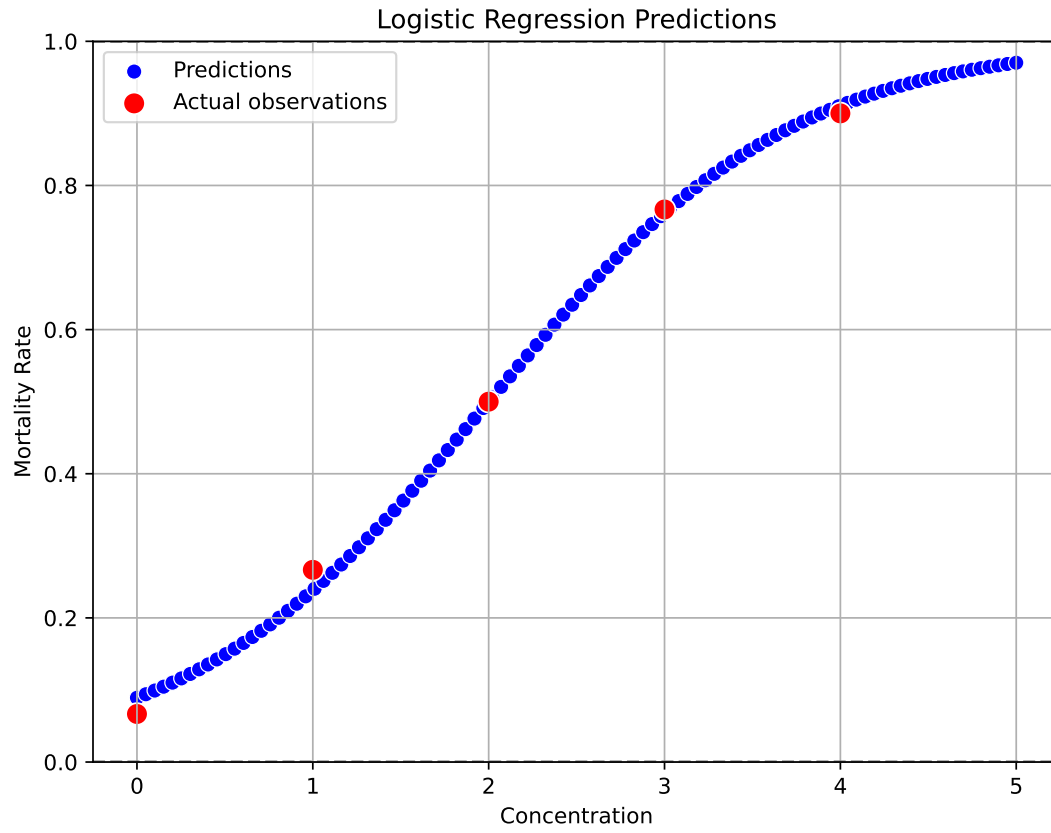
# Horizontal reference lines
plt.axhline(0, color = "gray", linestyle = "--")
plt.axhline(1, color = "gray", linestyle = "--")

# Labels and limits
plt.ylim(0, 1)
```

(0.0, 1.0)

```
plt.xlabel("Concentration")
plt.ylabel("Mortality Rate")
plt.title("Logistic Regression Predictions")

plt.legend()
plt.show()
```



3 Further on Binary and Binomial Data

3.1 Binary data: Logistic Regression

```
glm_insects_binary = smf.glm("survived_surgery ~ age + sex",
                             data = someSurgeryData,
                             family = sm.families.Binomial()).fit()
print(glm_insects_binary.summary())
```

3.2 Multinomial Data

```
## Load iris dataset
iris = load_iris()
df_iris = pd.DataFrame(iris.data, columns = iris.feature_names)
df_iris["Species"] = iris.target # Convert species to numerical labels

## Selecting the predictors and response variable
X = df_iris[["sepal length (cm)", "petal width (cm)"]] # Equivalent to Sepal.Length and Petal.Width
y = df_iris["Species"] # Target variable (0, 1, 2 for species)

## Fit multinomial logistic regression
```

```
multinom_iris = LogisticRegression(solver = "lbfgs", max_iter = 500)
multinom_iris.fit(X, y)
```

```
LogisticRegression(max_iter=500)
```

```
## Display model coefficients
print("Coefficients:\n", multinom_iris.coef_)
```

```
Coefficients:
[[-1.19275711 -3.80678258]
 [ 0.2789359  -0.07155451]
 [ 0.9138212   3.87833709]]
```

```
print("Intercept:\n", multinom_iris.intercept_)
```

```
Intercept:
[ 10.7105713  -0.10486227 -10.60570903]
```

3.3 Estimating the Performance of a Binary Model

```
## Read space- or tab-delimited file correctly
d_stab = pd.read_table("../Datasets/stability.dat", sep='\\s+', header = 0)

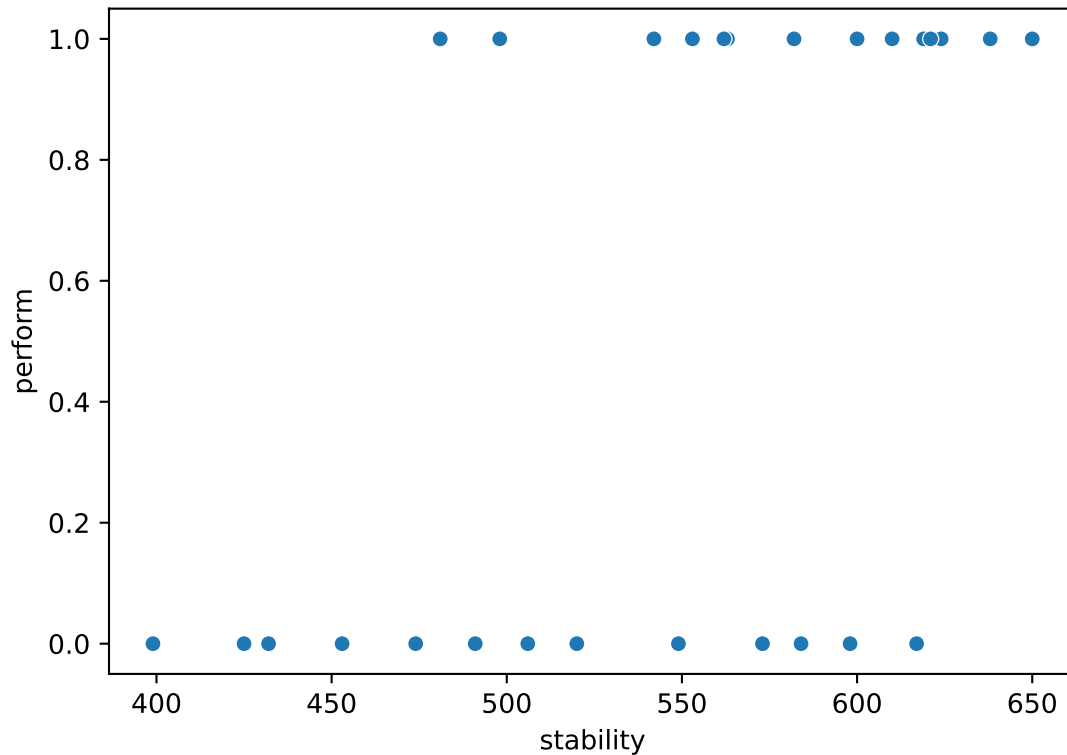
# Display structure equivalent to str(d.stab)
print(d_stab.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27 entries, 0 to 26
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   perform     27 non-null      float64
1   stability   27 non-null      float64
dtypes: float64(2)
memory usage: 564.0 bytes
None
```

```
print(d_stab.head())
```

```
   perform  stability
0      0.0      474.0
1      0.0      432.0
2      0.0      453.0
3      1.0      481.0
4      1.0      619.0
```

```
plt.clf()
sns.scatterplot(y = 'perform', x = 'stability', data = d_stab)
```



```
## Fit logistic regression (equivalent to method = "glm", family = "binomial" in ggplot)
glm_stab = smf.glm("perform ~ stability", data = d_stab,
                  family = sm.families.Binomial()).fit()

print(glm_stab.summary())
```

```

              Generalized Linear Model Regression Results
=====
Dep. Variable:          perform    No. Observations:          27
Model:                  GLM        Df Residuals:              25
Model Family:           Binomial   Df Model:                1
Link Function:           Logit     Scale:                 1.0000
Method:                  IRLS      Log-Likelihood:        -14.621
Date:                    Tue, 07 Oct 2025    Deviance:              29.242
Time:                    13:49:10    Pearson chi2:          25.0
No. Iterations:          4          Pseudo R-squ. (CS):    0.2606
Covariance Type:         nonrobust
=====

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	-10.3089	4.377	-2.355	0.019	-18.888	-1.730

stability	0.0189	0.008	2.402	0.016	0.003	0.034
=====						

```
# Generate predictions
stability_range = np.linspace(d_stab["stability"].min(), d_stab["stability"].max(), 100)
pred_data = pd.DataFrame({"stability": stability_range})
pred_data["perform_pred"] = glm_stab.predict(pred_data)

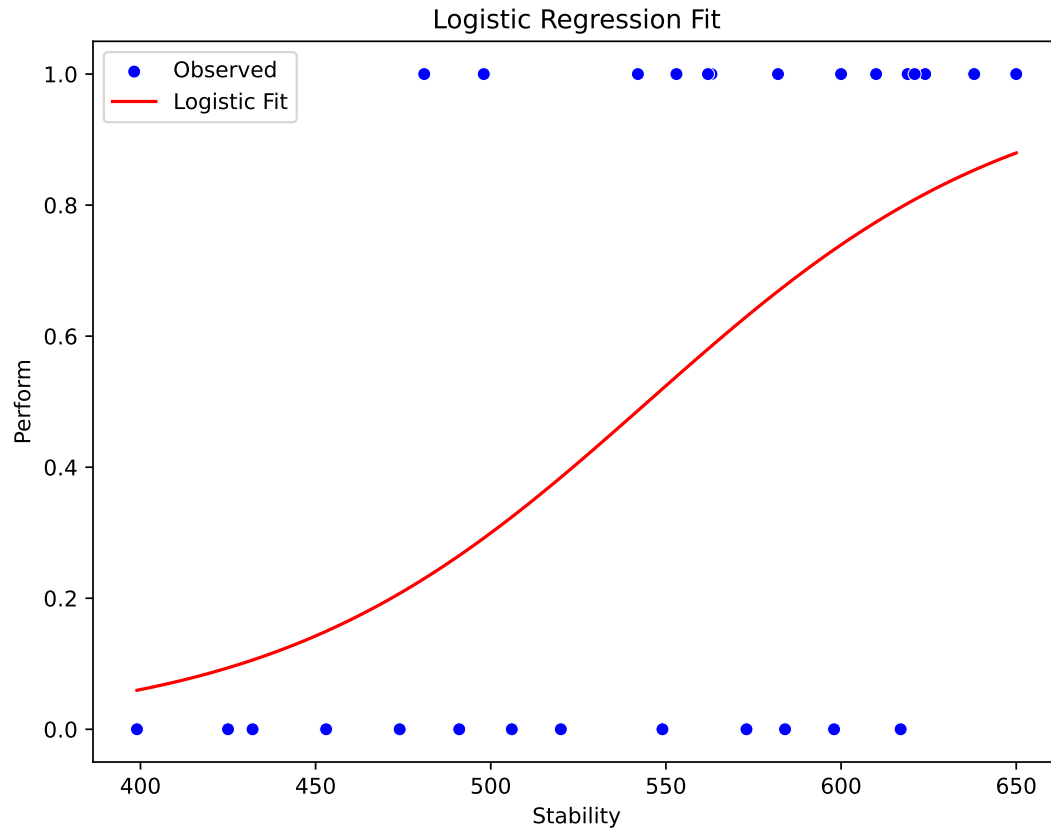
# Plot
plt.figure(figsize = (8, 6))

# Scatter plot (equivalent to geom_point())
sns.scatterplot(data = d_stab, x = "stability", y = "perform",
                color = "blue", label = "Observed")

# Logistic regression curve (equivalent to geom_smooth(method = "glm", family = "binomial"))
plt.plot(pred_data["stability"], pred_data["perform_pred"],
         color = "red", label = "Logistic Fit")

# Labels and title
plt.xlabel("Stability")
plt.ylabel("Perform")
plt.title("Logistic Regression Fit")

plt.legend()
```



```
fitted_stab = glm_stab.fittedvalues
##
binary_predictions = (fitted_stab >= 0.5).astype(int)
print(binary_predictions)
```

```
0    0
1    0
2    0
3    0
4    1
5    1
6    0
7    1
8    1
9    1
10   0
11   1
12   1
13   0
14   1
15   1
16   0
17   0
```

```

18    1
19    1
20    0
21    1
22    1
23    1
24    1
25    0
26    1
dtype: int64

```

```

##
d_obs_fit_stab = pd.DataFrame({'obs': d_stab['perform'], 'fit': binary_predictions})
print(d_obs_fit_stab.head())

```

```

      obs  fit
0  0.0    0
1  0.0    0
2  0.0    0
3  1.0    0
4  1.0    1

```

```

##
## Frequency count of a single categorical column
print(d_obs_fit_stab["obs"].value_counts())

```

```

obs
1.0    14
0.0    13
Name: count, dtype: int64

```

```

contingency_table = pd.crosstab(d_obs_fit_stab["obs"], d_obs_fit_stab["fit"])

## Display the contingency table
print(contingency_table)

```

```

fit  0    1
obs
0.0  8    5
1.0  3   11

```

```

prop_table = contingency_table.div(contingency_table.sum().sum())
print(round(prop_table, 2))

```

```

fit      0      1
obs
0.0  0.30  0.19
1.0  0.11  0.41

```

4 Appendix

4.1 Interpreting the Coefficients of a Poisson Model

```
## Load datasets (adjust the path if necessary)
esdcomp = pd.read_csv("../..../Datasets/esdcomp.csv")

## Fit Poisson GLM
glm_complaints = smf.glm("complaints ~ visits + residency + gender + revenue + hours",
                          data = esdcomp,
                          family = sm.families.Poisson()).fit()
print(glm_complaints.summary())
```

```

                        Generalized Linear Model Regression Results
=====
Dep. Variable:          complaints    No. Observations:          44
Model:                  GLM          Df Residuals:                38
Model Family:           Poisson     Df Model:                    5
Link Function:           Log         Scale:                      1.0000
Method:                 IRLS        Log-Likelihood:             -86.387
Date:                   Tue, 07 Oct 2025    Deviance:               49.995
Time:                   13:49:11           Pearson chi2:              50.3
No. Iterations:         5                Pseudo R-squ. (CS):       0.5921
Covariance Type:        nonrobust
=====

```

	coef	std err	z	P> z	[0.025	0.975]
Intercept	-0.0803	1.154	-0.070	0.945	-2.343	2.182
residency[T.Y]	-0.2320	0.203	-1.143	0.253	-0.630	0.166
gender[T.M]	0.1122	0.224	0.502	0.616	-0.326	0.550
visits	0.0009	0.000	2.806	0.005	0.000	0.002
revenue	-0.0034	0.004	-0.814	0.416	-0.012	0.005
hours	-0.0002	0.001	-0.237	0.813	-0.001	0.001

```
=====
```

4.1.1 Interpreting the Coefficients of a Poisson Model: Factors

```
## Extract coefficient for sexM
coef_genderM = glm_complaints.params["gender[T.M]"]
print("Coefficient for gender M:", coef_genderM)
```

Coefficient for gender M: 0.11223911512012463

```
## Exponentiated coefficient
exp_coef_genderM = np.exp(coef_genderM).round(2)
print("Exponentiated coefficient for sexM:", exp_coef_genderM)
```

Exponentiated coefficient for sexM: 1.12

```
## First doctor data
first_doctor = esdcomp[0:1]
print("First doctor:", first_doctor)
```

```
First doctor:   visits  complaints residency gender  revenue    hours
0    2014         2           Y      F   263.03  1287.25
```

```
## Fitted value for first doctor
fitted_first_doctor = glm_complaints.fittedvalues[0:1]
print("Fitted complaints for first doctor:", fitted_first_doctor)
```

```
Fitted complaints for first doctor: 0    1.663748
dtype: float64
```

```
## Compute predicted complaints if the first doctor was male
first_doctor_as_male = first_doctor
first_doctor_as_male.iloc[0, 3] = "M"
print(first_doctor_as_male)
```

```
   visits  complaints residency gender  revenue    hours
0    2014         2           Y      M   263.03  1287.25
```

```
pred_first_doc_male = glm_complaints.predict(first_doctor_as_male)
print("Predicted complaints if first doctor was male:", pred_first_doc_male)
```

```
Predicted complaints if first doctor was male: 0    1.861369
dtype: float64
```

```
## Manually computing expected complaints for first doctor as male
manual_pred = fitted_first_doctor * np.exp(coef_genderM)
print("Manually computed complaints for first doctor as male:", manual_pred)
```

```
Manually computed complaints for first doctor as male: 0    1.861369
dtype: float64
```

4.1.2 Interpreting the Coefficients of a Poisson Model: Continuous Predictors

```
## Exponentiate coefficient for visits
exp_coef_visits = np.exp(glm_complaints.params["visits"])
print("Exponentiated coefficient for visits:", exp_coef_visits.round(5))
```

```
Exponentiated coefficient for visits: 1.00095
```

```
## Range of visits
visits_range = (esdcomp["visits"].min(), esdcomp["visits"].max())
print("Range of visits:", visits_range)
```

```
Range of visits: (np.int64(879), np.int64(3763))
```

```
## Increase number of visits by 50
coef_visits_50_add = glm_complaints.params["visits"] * 50
print(coef_visits_50_add.round(5))
```

0.0475

```
## Exponentiate it
exp_coef_visits_50_add = np.exp(coef_visits_50_add)
print("Exponentiated coefficient for visits * 50:", exp_coef_visits_50_add.round(5))
```

Exponentiated coefficient for visits * 50: 1.04864

4.2 Interpreting the Coefficients of a Binomial Model

```
plt.clf()
# Generate new data for prediction
new_data = pd.DataFrame({"conc": np.linspace(0, 5, 100)})
new_data["pred_insects"] = glm_insects.predict(new_data)

# Plot
plt.figure(figsize = (8, 6))
plt.grid()

# Plot predictions
sns.scatterplot(x = new_data["conc"], y = new_data["pred_insects"],
               color = "blue", s = 50, label = "Predictions")

# Plot actual observations on top
sns.scatterplot(x = bliss["conc"], y = bliss["mortality_rate"],
               color = "red", s = 100, label = "Actual observations")

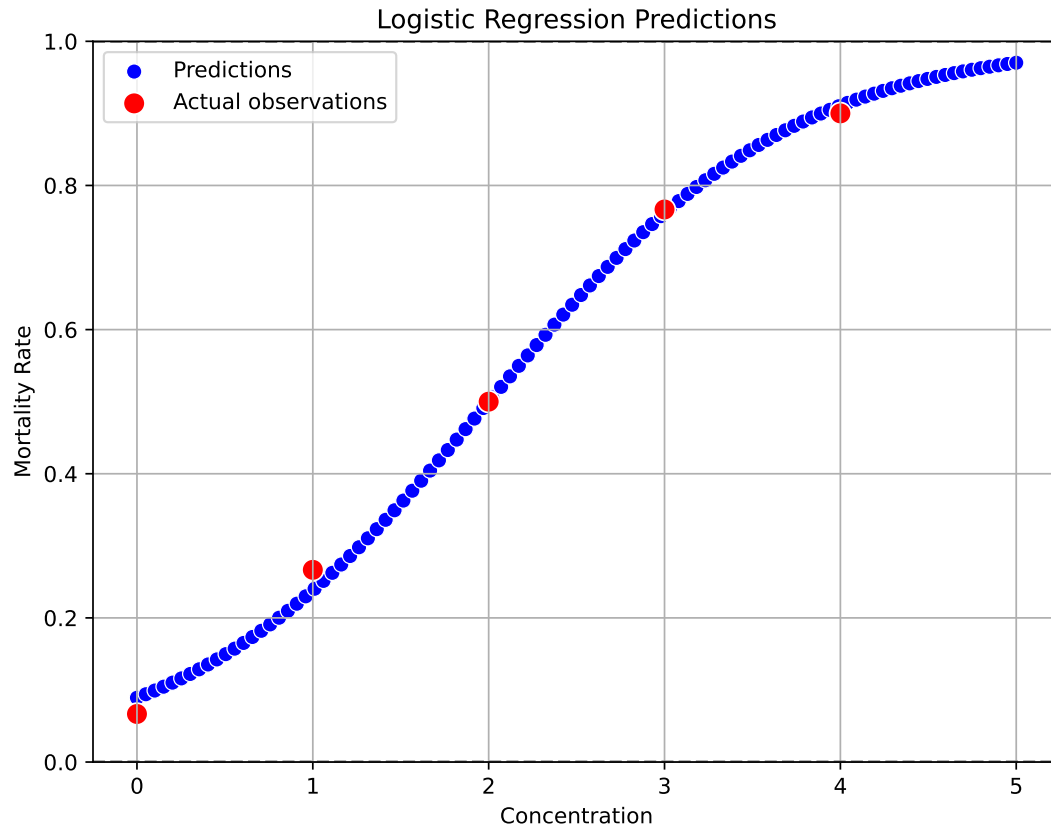
# Horizontal reference lines
plt.axhline(0, color = "gray", linestyle = "--")
plt.axhline(1, color = "gray", linestyle = "--")

# Labels and limits
plt.ylim(0, 1)
```

(0.0, 1.0)

```
plt.xlabel("Concentration")
plt.ylabel("Mortality Rate")
plt.title("Logistic Regression Predictions")

plt.legend()
```



```
print(glm_insects.params)
```

```
Intercept    -2.323790  
conc          1.161895  
dtype: float64
```

```
##  
print(np.exp(glm_insects.params['conc']).round(5))
```

```
3.19598
```

```

# Generate new data for prediction
new_data = pd.DataFrame({"conc": np.linspace(0, 5, 100)})
new_data["pred_insects"] = glm_insects.predict(new_data)

# Plot
plt.figure(figsize = (8, 6))
plt.grid()

# Plot predictions
sns.scatterplot(x = new_data["conc"], y = new_data["pred_insects"],
                color = "blue", s = 50, label = "Predictions")

# Plot actual observations on top
sns.scatterplot(x = bliss["conc"], y = bliss["mortality_rate"],
                color = "red", s = 100, label = "Actual observations")

# Horizontal reference lines
plt.axhline(0, color = "gray", linestyle = "--")
plt.axhline(1, color = "gray", linestyle = "--")

# Labels and limits
plt.ylim(0, 1)

```

```

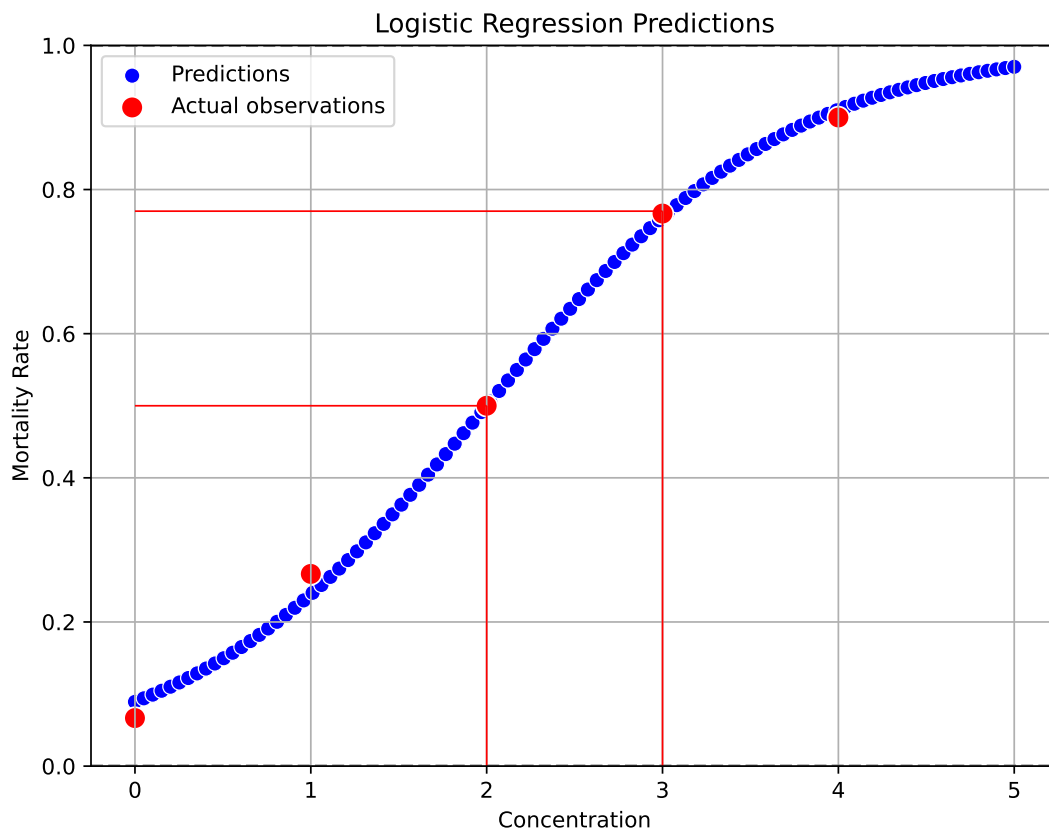
(0.0, 1.0)

```



```
plt.xlabel("Concentration")
plt.ylabel("Mortality Rate")
plt.title("Logistic Regression Predictions")

plt.legend()
plt.vlines(x = 2, ymin = 0, ymax = 0.5, color = 'red', linewidth = 0.8)
plt.hlines(y = 0.5, xmin = 0, xmax = 2, color = 'red', linewidth = 0.8)
##
plt.vlines(x = 3, ymin = 0, ymax = 0.77, color = 'red', linewidth = 0.8)
plt.hlines(y = 0.77, xmin = 0, xmax = 3, color = 'red', linewidth = 0.8)
```



4.2.1 Interpreting the Coefficients of a Binomial Model: Factors

```
bins = [-np.inf, 0, 2, 4]
labels = ["no insecticide", "low conc", "high conc"]

# Apply binning (equivalent to cut() in R)
bliss["conc_asFactor"] = pd.cut(bliss["conc"], bins=bins, labels=labels)

# Display factor levels (unique categories)
print(bliss["conc_asFactor"].cat.categories)
```

```
Index(['no insecticide', 'low conc', 'high conc'], dtype='object')
```

```
pd.crosstab(bliss["conc"], bliss["conc_asFactor"])
```

```
conc_asFactor  no insecticide  low conc  high conc
conc
0                1            0            0
1                0            1            0
2                0            1            0
3                0            0            1
4                0            0            1
```

```
glm_insects_fac = smf.glm("mortality_rate ~ conc_asFactor",
                          data = bliss,
                          family = sm.families.Binomial()).fit()
print(glm_insects_fac.summary())
```

Generalized Linear Model Regression Results

```
=====
Dep. Variable:          mortality_rate    No. Observations:          5
Model:                  GLM              Df Residuals:              2
Model Family:           Binomial         Df Model:                  2
Link Function:           Logit           Scale:                    1.0000
Method:                  IRLS            Log-Likelihood:             -1.7205
Date:                    Tue, 07 Oct 2025 Deviance:                  0.18210
Time:                    13:49:12         Pearson chi2:               0.179
No. Iterations:          5               Pseudo R-squ. (CS):        0.3265
Covariance Type:         nonrobust
=====
```

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept          -2.6391      4.009     -0.658      0.510     -10.496      5.218
conc_asFactor[T.low conc]    2.1636      4.265      0.507      0.612      -6.195     10.522
conc_asFactor[T.high conc]   4.2485      4.435      0.958      0.338      -4.444     12.941
=====
```

```
## smf.glm doesn't directly support quasi poisson models
```